

基于连通性状态压缩的动态规划问题

长沙市雅礼中学 陈丹琦

【摘要】

基于状态压缩的动态规划问题是一类以集合信息为状态且状态总数为指数级的特殊的动态规划问题。在状态压缩的基础上，有一类问题的状态中必须要记录若干个元素的连通情况，我们称这样的问题为基于连通性状态压缩的动态规划问题，本文着重对这类问题的解法及优化进行探讨和研究。

本文主要从动态规划的几个步骤——划分阶段，确立状态，状态转移以及程序实现来介绍这类问题的一般解法，会特别针对到目前为止信息学竞赛中涌现出来的几类题型的解法作一个探讨。结合例题，本文还会介绍作者在减少状态总数和降低转移开销两个方面对这类问题优化的一些心得。

【关键词】

状态压缩 连通性 括号表示法 轮廓线 插头 棋盘模型

【目录】

【序言】	3
【正文】	5
一. 问题的一般解法.....	5
【例 1】 Formula 1	5
问题描述.....	5
算法分析.....	5
小结.....	11
二. 一类简单路径问题.....	11
【例 2】 Formula 2	14
问题描述.....	14
算法分析.....	15
小结.....	16
三. 一类棋盘染色问题.....	16
【例 3】 Black & White	16
问题描述.....	17
算法分析.....	17
小结.....	19
四. 一类基于非棋盘模型的问题.....	19
【例 4】 生成树计数	19
问题描述.....	19
算法分析.....	19
小结.....	20
五. 一类最优性问题的剪枝技巧.....	20
【例 5】 Rocket Mania	20
问题描述.....	21
算法分析.....	21
小结.....	23
六. 总结.....	23
【参考文献】	23
【感谢】	24
【附录】	24

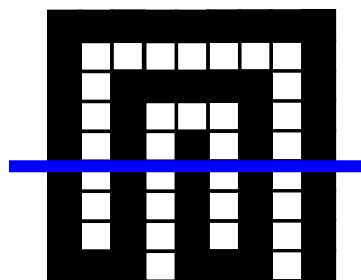
【序言】

先看一个非常经典的问题——旅行商问题(即 TSP 问题, Traveling Salesman Problem): 一个 $n(\leq 15)$ 个点的带权完全图, 求权和最小的经过每个点恰好一次的封闭回路. 这个问题已经被证明是 NP 完全问题, 那么对于这样一类无多项式算法的问题, 搜索算法是不是解决问题的唯一途径呢? 答案是否定的. 不难发现任何时候我们只需要知道哪些点已经被遍历过而遍历点的具体顺序对以后的决策是没有影响的, 因此不妨以当前所在的位置 i , 遍历过的点的集合 S 为状态作动态规划:

$$f(i, S) = \min\{f(j, S - \{i\}) + \text{dist}(j, i)\}, \text{ 其中 } j < i, i, j \in S.$$

动态规划的时间复杂度为 $O(2^n * n^2)$, 虽然为指数级算法, 但是对于 $n = 15$ 的数据规模来说已经比朴素的 $O(n!)$ 的搜索算法高效很多了. 我们通常把这样一类以一个集合内的元素信息作为状态且状态总数为指数级别的动态规划称为**基于状态压缩的动态规划或集合动态规划**. 基于状态压缩的动态规划问题通常具有以下两个特点: 1. 数据规模的某一维或几维非常小; 2. 它需要具备动态规划问题的两个基本性质: **最优性原理**和**无后效性**.

一般的状态压缩问题, 压缩的是一个较小范围内每个元素的决策, 状态中元素的信息相对独立. 而有些问题, 仅仅记录每个元素的决策是不够的, 不妨再看一个例子: 给你一个 $m * n$ ($m, n \leq 9$) 的矩阵, 每个格子有一个价值 $V_{i,j}$, 要求找一个连通块使得该连通块内所有格子的价值之和最大. 按从上到下的顺序依次考虑每个格子选还是不选, 下图为一个极端情况, 其中黑色的格子为所选的连通块. 只考虑前 5 行的时候, 所有的黑色格子形成了三个连通块, 而最后所有的黑色格子形成一个连通块. 如果状态中只单纯地记录前一行或前几行的格子选还是不选, 是无法准确描述这个状态的, 因此压缩的状态中我们需要增加一维, 记录若干个格子之间的连通情况. 我们把这一类必须要在状态中记录若干个元素之间的连通信息的问题称为**基于连通性状态压缩的动态规划问题**. 本文着重对这类问题进行研究.



连通是图论中一个非常重要的概念, 在一个无向图中, 如果两个顶点之间存在一条路径, 则称这两个点连通. 而基于连通性状态压缩的动态规划问题与图论模型有着密切的关联, 比如后文涉及到的哈密顿回路、生成树等等. 通常这类问题的本身与连通性有关或者隐藏着连通信息.

全文共有六个章节.

第一章，问题的一般解法，介绍解决基于连通性状态压缩的动态规划问题的一般思路和解题技巧；

第二章，一类简单路径问题，介绍一类基于棋盘模型的简单路径问题的状态表示的改进——括号表示法以及提出广义的括号表示法；

第三章，一类棋盘染色问题，介绍解决一类棋盘染色问题的一般思路；

第四章，一类基于非棋盘模型的问题，介绍解决一类非棋盘模型的连通性状态压缩问题的一般思路；

第五章，一类最优性问题的剪枝技巧，本章的重点是优化，探讨如何通过剪枝来减少扩展的状态的总数从而提高算法的效率；

第六章，总结，回顾前文，总结解题方法.

【正文】

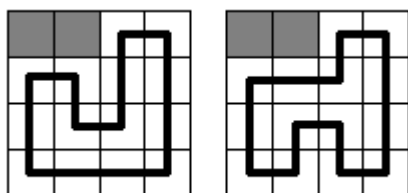
一. 问题的一般解法

基于连通性状态压缩的动态规划问题通常具有一个比较固定的模式，几乎所有的题目都是在这个模式的基础上变形和扩展的。本章选取了一个有代表性的例题来介绍这一类问题的一般解法。

【例 1】Formula 1¹

问题描述

给你一个 $m * n$ 的棋盘，有的格子是障碍，问共有多少条回路使得经过每个非障碍格子恰好一次。 $m, n \leq 12$ 。

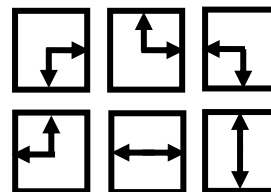


如图， $m = n = 4$ ， $(1, 1), (1, 2)$ 是障碍，共有 2 条满足要求的回路。

算法分析

【划分阶段】 这是一个典型的基于**棋盘模型**的问题，棋盘模型的特殊结构，使得它成为连通性状态压缩动态规划问题最常见的“舞台”。通常来说，棋盘模型有三种划分阶段的方法：逐行，逐列，逐格。顾名思义，逐行即从上到下或从下到上依次考虑每一行的状态，并转移到下一行；逐列即从左到右或从右到左依次考虑每一列的状态，并转移到下一列；逐格即按一定的顺序(如从上到下，从左到右)依次考虑每一格的状态，并转移到下一个格子。

对于本题来说，逐行递推和逐列递推基本类似²，接下来我们会对逐行递推和逐格递推的状态确立，状态转移以及程序实现一一介绍。

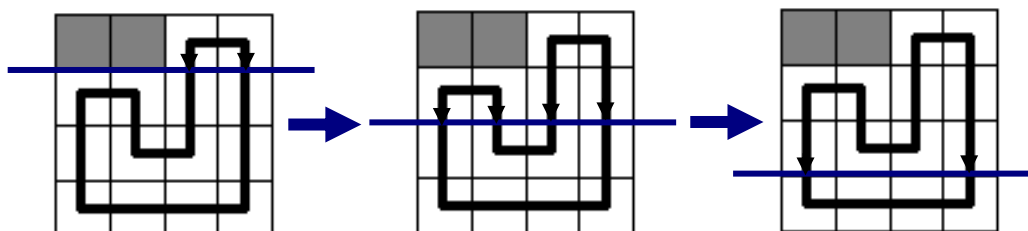
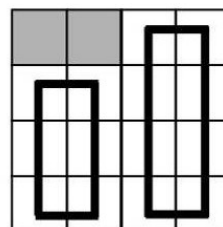


¹ Ural1519, Timus Top Coders : Third Challenge

² 有的题目，逐行递推和逐列递推的状态表示有较大的区别，比如本文后面会讲到的 Rocket Mania 一题

【确立状态】 先提出一个非常重要的概念——“插头”。对于一个 4 连通的问题来说，它通常有上下左右 4 个插头，一个方向的插头存在表示这个格子在这个方向可以与外面相连。本题要求回路的个数，观察可以发现所有的非障碍格子一定是从一个格子进来，另一个格子出去，即 4 个插头恰好有 2 个插头存在，共 6 种情况。

逐行递推 不妨按照从上到下的顺序依次考虑每一行。分析第 i 行的哪些信息对第 $i+1$ 行有影响：我们需要记录第 i 行的每个格子是否有下插头，这决定了第 $i+1$ 行的每个格子是否有上插头。仅仅记录插头是否存在是不够的，可能导致出现多个回路（如右图），而本题要求一个回路，也就隐含着最后所有的非障碍格子通过插头连接成了一个连通块，因此还需要记录第 i 行的 n 个格子的连通情况。



插头：0011

连通性：(3,4)

插头：1111

连通性：(1,2) (3,4)

插头：1001³

连通性：(1,2,3,4)⁴

我们称图中的蓝线为轮廓线，任何时候只有轮廓线上方与其直接相连的格子和插头才会对轮廓线以下的格子产生直接的影响。通过上面的分析，可以写出动态规划的状态： $f(i, S_0, S_1)$ 表示前 i 行，第 i 行的 n 个格子是否具有下插头的 n 位的二进制数为 S_0 ，第 i 行的 n 个格子之间的连通性为 S_1 的方案总数。

如何表示 n 个格子的连通性呢？通常给每一个格子标记一个正数，属于同一个的连通块的格子标记相同的数。比如 $\{1,1,2,2\}$ 和 $\{2,2,1,1\}$ 都表示第 1,2 个格子属于一个连通块，第 3,4 个格子属于一个连通块。为了避免出现同一个连通信息有不同的表示，一般会使用最小表示法。

一种最小表示法为：所有的障碍格子标记为 0，第一个非障碍格子以及与其连通的所有格子标记为 1，然后再找第一个未标记的非障碍格子以及与其连通的格子标记为 2，……，重复这个过程，直到所有的格子都标记完毕。比如连通信息 $((1,2,5),(3,6),(4))$ 表示为 $\{1,1,2,3,1,2\}$ 。还有一种最小表示法，即一个连通块内所有的格子都标记成该连通块最左边格子的列编号，比如上面这个例子，我们表

³ 从左到右，0 表示无插头，1 表示有插头

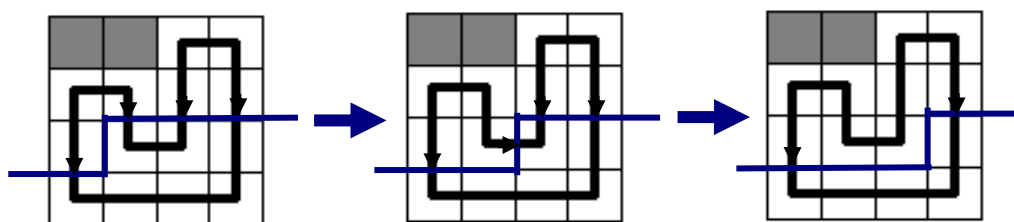
⁴ 括号内的数表示的是格子的列编号，一个括号内的格子属于一个连通块

示为 $\{1,1,3,4,1,3\}$. 两种表示方法在转移的时候略有不同, 本文后面将会提到⁵. 如上图三个状态我们可以依次表示为 $f(1, (0011)_2, \{0, 0, 1, 1\})$, $f(2, (1111)_2, \{1, 1, 2, 2\})$, $f(3, (1001)_2, \{1, 1, 1, 1\})$.

状态表示的优化 通过观察可以发现如果轮廓线上方的 n 个格子中某个格子没有下插头，那么它就不会再与轮廓线以下的格子直接相连，它的连通性对轮廓线以下的格子不会再有影响，也就成为了“冗余”信息。不妨将记录格子的连通性改成记录插头的连通性，如果这个插头存在，那么就标记这个插头对应的格子的连通标号，如果这个插头不存在，那么标记为 0。这样状态就从 $f(i, S_0, S_1)$ 精简为 $f(i, S)$ ，上图三个状态表示为 $f(1, \{0, 0, 1, 1\})$ ， $f(2, \{1, 1, 2, 2\})$ ， $f(3, \{1, 0, 0, 1\})$ 。优化后不仅状态表示更加简单，而且状态总数将会大大减少。

逐格递推 按照从上到下, 从左到右的顺序依次考虑每一格. 分析转移完 (i, j) 这个格子后哪些信息对后面的决策有影响: 同样我们可以刻画出轮廓线, 即轮廓线上方是已决策格子, 下方是未决策格子. 由图可知与轮廓线直接相连的格子有 n 个, 直接相连的插头有 $n+1$ 个, 包括 n 个格子的下插头以及 (i, j) 的右插头. 为了保持轮廓线的“连贯性”, 不妨从左到右依次给 n 个格子标号, $n+1$ 个插头标号. 类似地, 我们需要记录与轮廓线直接相连的 $n+1$ 个插头是否存在以及 n 个格子的连通情况.

通过上面的分析，很容易写出动态规划的状态： $f(i, j, S_0, S_1)$ 表示当前转移完 (i, j) 这个格子， $n+1$ 个插头是否存在表示成一个 $n+1$ 位的二进制数 S_0 ，以及 n 个格子的连通性为 S_1 的方案总数。


$$f(3,1,(10111),\{1,1,2,2\}) \quad f(3,2,(10111),\{1,1,2,2\}) \quad f(3,3,(10001),\{1,1,1,1\})$$

逐行递推的时候我们提到了状态的优化，同样地，我们也可以把格子的连通性记录在插头上，新的状态为 $f(i, j, S)$ ，上图 3 个状态依次为 $f(3, 1, \{1, 0, 1, 2, 2\})$ ， $f(3, 2, \{1, 0, 1, 2, 2\})$ ， $f(3, 3, \{1, 0, 0, 0, 1\})$ 。

⁵因为第一种表示法更加直观, 本文如果不作特殊说明, 默认使用第一种最小表示法

【转移状态】

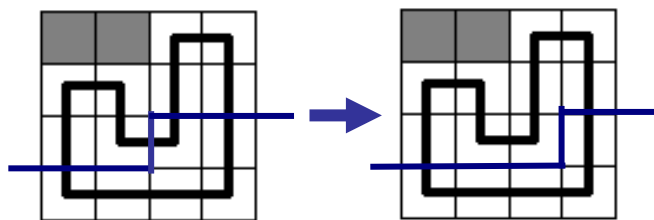
状态的转移开销主要包含两个方面：每个状态转移的状态数，计算新的状态的时间。

逐行递推 假设从第 i 行转移到第 $i+1$ 行，我们需要枚举第 $i+1$ 行的每个格子的状态(共 6 种情况)，对于任何一个非障碍格子，它是否有上插头和左插头已知，因此最多只有 2 种情况，状态的转移数 $\leq 2^n$ 。

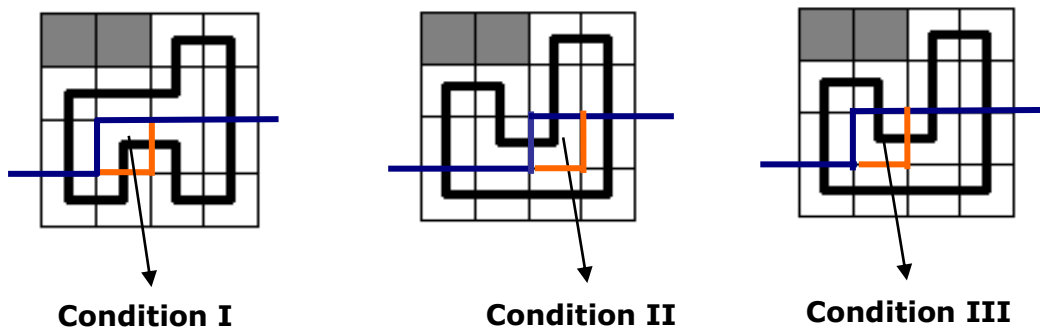
枚举完第 $i+1$ 行每个格子的状态后，需要计算第 $i+1$ 行 n 个格子之间的连通性的最小表示，通常可以使用并查集的 **Father** 数组对其重新标号或者重新执行一次 BFS/DFS，时间复杂度为 $O(n)$ ，最后将格子的连通性转移到插头的连通性上。

特别需要注意的是在转移的过程中，为了避免出现多个连通块，除了最后一行，任何时候一个连通分量内至少有一个格子有下插头。

逐格递推 仔细观察下面这个图，当 $f(i, j-1, S)$ 转移到 $f(i, j, S')$ 时，轮廓



线上 n 个格子只有 $(i-1, j)$ 被改成 (i, j) ， $n+1$ 个插头只有 2 个插头被改动，即 $(i, j-1)$ 的右插头修改成 (i, j) 的下插头和 $(i-1, j)$ 的下插头修改成 (i, j) 的右插头。转移的时候枚举 (i, j) 的状态分情况讨论。一般棋盘模型的逐格递推转移有 3 类情况：新建一个连通分量，合并两个连通分量，以及保持原来的连通分量。下面针对本题进行分析：



情况 1 新建一个连通分量，这种情况出现在 (i, j) 有右插头和下插头。新建的两个插头连通且不与其它插头连通，这种情况下需要将这两个插头连通分量标号标记成一个未标记过的正数，重新 $O(n)$ 扫描保证新的状态满足最小表示。

情况 2 合并两个连通分量，这种情况出现在 (i, j) 有上插头和左插头。如果两个插头不连通，那么将两个插头所处的连通分量合并，标记相同的连通块标号， $O(n)$ 扫描保证最小表示；如果已经连通，相当于出现了一个回路，这种情况只能出现在最后一个非障碍格子。

情况 3 保持原来的连通分量，这种情况出现在 (i, j) 的上插头和左插头恰好有一个，下插头和右插头也恰好有一个。下插头或右插头相当于是左插头或上插头的延续，连通块标号相同，并且不会影响到其他的插头的连通块标号，计算新的状态的时间为 $O(1)$ 。

注意当从一行的最后一个格子转移到下一行的第一个格子的时候，轮廓线需要特殊处理。值得一提的是，上面三种情况计算新的状态的时间分别为 $O(n)$, $O(n)$, $O(1)$ ，如果使用前面提到的第二种最小表示方法，情况 1 只需要 $O(1)$ ，但是情况 3 可能需要 $O(n)$ 重新扫描。

比较一下逐行递推和逐格递推的状态的转移，逐行递推的每一个转移的状态总数为指数级，而逐格递推为 $O(1)$ ，每次计算新的状态的时间两者最坏情况都为 $O(n)$ ，但是逐行递推的常数要比逐格递推大，从转移开销这个角度来看，逐格递推的优势是毋庸置疑的。

【程序实现】

逐行递推和逐格递推的程序实现基本一致，下面以逐格递推为例来说明。首先必须解决的一个问题是，对于像 $f(3, 2, \{1, 0, 1, 2, 2\})$ 这样的一个状态我们该如何存储，可以开一个长度为 $n+1$ 的数组来存取 $n+1$ 个插头的连通性，但是数组判重并不方便，而且空间较大。不妨将 $n+1$ 个元素进行**编码**，用一个或几个整数来存储，当我们需要取一个状态出来对它进行修改的时候再进行**解码**。

编码最简单的方法就是表示成一个 $n+1$ 位的 p 进制数， p 可以取能够达到的最大的连通块标号加 1^6 ，对本题来说，最多出现 $\lfloor n/2 \rfloor \leq 6$ 个连通块，不妨取 $p = 7$ 。在不会超过数据类型的范围的前提下，建议将 p 改成 2 的幂，因为位运算比普通的运算要快很多，本题最好采用 8 进制来存储。

如需大范围修改连通块标号，最好将状态 $O(n)$ 解码到一个数组中，修改后再 $O(n)$ 计算出新的 p 进制数，而对于只需要局部修改几个标号的情况下，可以直接用 $(x \div p^{i-1}) \bmod p$ 来获取第 i 位的状态，用 $\pm k * p^{i-1}$ 直接对第 i 位进行修改。

最后我们探讨一下实现的方法，一般有两种方法：

1. 对所有可能出现的状态进行编码，枚举编码方式：预处理将所有可能的连通性状态搜索出来，依次编号 $1, 2, 3, \dots, Tot$ ，那么状态为 $f(i, j, k)$ 表示转移完

⁶ 因为还要把 0 留出来存没有插头的情况

(i, j) 后轮廓线状态编号为 k 的方案总数. 将所有状态存入 Hash 表中, 使得每个状态与编号一一对应, 程序框架如下:

```

For  $i \leftarrow 1$  to  $m$ 
  For  $j \leftarrow 1$  to  $n$ 
    For  $k \leftarrow 1$  to  $Tot$ 
      For  $x \leftarrow (i, j, State[k])$  的所有转移后的状态
         $k' \leftarrow$  状态  $x$  的编号
         $f(i', j', k') \leftarrow f(i', j', k') + f(i, j, k)$ ,  $(i', j')$  为  $(i, j)$  的后继格子.
      End For
    End For
  End For
End For

```

2. 记忆化宽度优先搜索: 将初始状态放入队列中, 每次取队首元素进行扩展, 并用 Hash 对扩展出来的新的状态判重. 程序框架如下:

```

Queue.Push(所有初始状态)
While not Empty(Queue)
   $p \leftarrow$  Queue.Pop()
  For  $x \leftarrow p$  的所有转移后的状态
    If  $x$  之前扩展过 Then
      Sum [ $x$ ]  $\leftarrow$  Sum [ $x$ ] + Sum [ $p$ ]
    Else
      Queue.Push( $x$ )
      Sum [ $x$ ]  $\leftarrow$  Sum [ $p$ ]
    End If
  End For
End While

```

比较上述两种实现方法, 直接编码的方法实现简单, 结构清晰, 但是有一个很大的缺点: 无效状态可能很多, 导致了很多次空循环, 而大大影响了程序的效率. 下面是一组实验的比较数据:

表 1. 直接编码与宽度优先搜索扩展状态总数比较

测试数据	宽度优先搜索 扩展状态总数	直接编码 Tot	$Tot * m * n$	无效状态比率
$m = 9, n = 9$ (1,1)为障碍	30930	2188	177228	82.5%
$m = 10, n = 10$ 无障碍	134011	5798	579800	76.8%
$m = 11, n = 11$ (1,1)为障碍	333264	15511	1876831	82.2%
$m = 12, n = 12$ 无障碍	1333113	41835	6024240	77.9%

可以看出直接编码扩展的无效状态的比率非常高,对于障碍较多的棋盘其对比更加明显,因此通常来说宽度优先搜索扩展比直接编码实现效率要高.

Hash 判重的优化: 使用一个 HashSize 较小的 Hash 表,每转移一个 (i,j) 清空一次,每次判断状态 x 是否扩展过的程序效率比用一个 HashSize 较大的 Hash 表每次判断状态 (i,j,x) 高很多.类似地,在不需要记录路径的情况下,也可以使用滚动的扩展队列来代替一个大的扩展队列.

最后我们比较一下,不同的实现方法对程序效率的影响⁷:

Program 1 : 8-Based, 枚举编码方式.

Program 2 : 8-Based, 队列扩展, HashSize = 3999997.

Program 3 : 8-Based, 队列扩展, HashSize = 4001, Hash 表每次清空.

Program 4 : 7-Based, 队列扩展, HashSize = 4001, Hash 表每次清空.

表 2. 不同的实现方法的程序效率的比较

测试数据	Program 1	Program 2	Program 3	Program 4
$m = 10, n = 10$ 无障碍棋盘	46ms	31ms	15ms	31ms
$m = 11, n = 11$ (1,1)为障碍	140ms	499ms	109ms	187ms
$m = 12, n = 12$ 无障碍	624ms	1840ms	499ms	873ms

小结

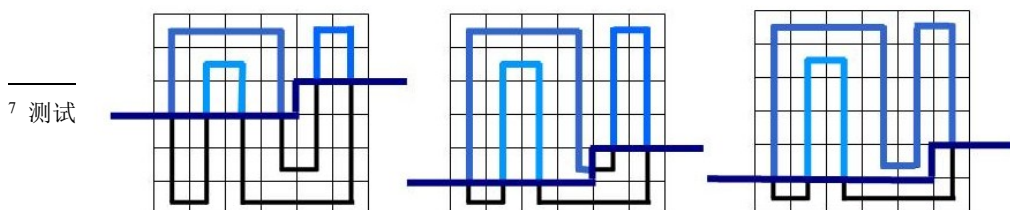
本章从划分阶段,确立状态,状态转移以及程序实现四个方面介绍了基于连通性状态压缩动态规划问题的一般解法,并在每个方面归纳了一些不同的方法,最后对不同的算法的效率进行比较.在平时的解题过程中我们要学会针对题目的特点和数据规模“对症下药”,选择最合适的方法而达到最好的效果.

由于逐格递推的转移开销比逐行递推小很多,下文如果不作特殊说明,我们都采用逐格的阶段划分.

二. 一类简单路径问题

这一章我们会针对一类基于棋盘模型的简单回路和简单路径问题的解法作一个探讨.简单路径,即除了起点和终点可能相同外,其余顶点均不相同的路径,而简单回路为起点和终点相同的简单路径. Formula 1 是一个典型的棋盘模型的简单回路问题,这一章我们继续以这个题为例来说明.

首先我们分析一下简单回路问题有什么特点:



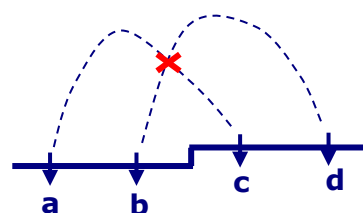
仔细观察上面的图，可以发现轮廓线上方是由若干条互不相交的路径构成的，而每条路径的两个端口恰好对应了轮廓线上的两个**插头**！一条路径上的所有格子对应的是一个连通块，而每条路径的两个端口对应的两个插头是连通的而且不与其他任何一个插头连通。

在上一章我们提到了逐格递推转移的时候的三种情况：新建一个连通分量，合并两个连通分量，保持原来的连通分量，它们分别等价于两个插头成为了一条新的路径的两端，两条路径的两个端口连接起来形成一条更长的路径或一条路径的两个端口连接起来形成一个回路以及延长原来的路径。

通过上面的分析我们知道了简单回路问题一定满足任何时候轮廓线上每一个连通分量恰好有 2 个插头，那么这些插头之间有什么性质呢？

【性质】轮廓线上从左到右 4 个插头 a, b, c, d ，如果 a, c 连通，并且与 b 不连通，那么 b, d 一定不连通。

证明：反证法，如果 a, c 连通， b, d 连通，那么轮廓线上方一定至少存在一条 a 到 c 的路径和一条 b 到 d 的路径。如图，两条路径一定会有交点，不妨设两条路径相交于格子 P ，那么 P 既与 a, c 连通，又与 b, d 连通，可以推出 a, c 与 b, d 连通，矛盾，得证。



这个性质对所有的棋盘模型的问题都适用。

“两两匹配”，“不会交叉”这样的性质，我们很容易联想到**括号匹配**。将轮廓线上每一个连通分量中左边那个插头标记为左括号，右边那个插头标记为右括号，由于插头之间不会交叉，那么左括号一定可以与右括号一一对应。这样我们就可以使用 3 进制——0 表示无插头，1 表示左括号插头，2 表示右括号插头记录下所有的轮廓线信息。不妨用 # 表示无插头，那么上面的三幅图分别对应的是 $((0)\#(0))$, $((0\#)(0))$, $((0)\#\#\#)$ ，即 $(1122012)_3, (1120212)_3, (1120002)_3$ ，我们称这种状态的表示方法为**括号表示法**。

依然分三类情况来讨论状态的转移：

为了叙述方便，不妨称 $(i, j-1)$ 的右插头为 p ， $(i-1, j)$ 的下插头为 q ， (i, j) 的下插头为 p' ，右插头为 q' ，那么每次转移相当于轮廓线上插头 p 的信息修改成 p' 的信息，插头 q 的信息修改成 q' 的信息，设 $W(x) = 0, 1, 2$ 表示插头 x 的状态。

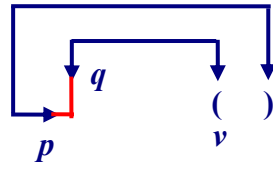
情况 1 新建一个连通分量，这种情况下 $W(p) = 0$ ， $W(q) = 0$ ， p' ， q' 两个

插头构建了一条新的路径，相当于 p' 为左括号， q' 为右括号，即 $W(p') \leftarrow 1$ ， $W(q') \leftarrow 2$ ，计算新的状态的时间为 $O(1)$ 。

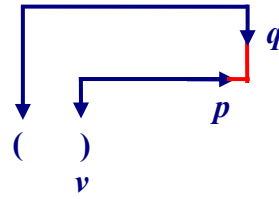
情况 2 合并两个连通分量，这种情况下 $W(p) > 0$ ， $W(q) > 0$ ， $W(p') \leftarrow 0$ ， $W(q') \leftarrow 0$ ，根据 p, q 为左括号还是右括号分四类情况讨论：

情况 2.1 $W(p) = 1$ ， $W(q) = 1$ 。那么需要将 q 这个左括号与之对应的右括号 v 修改成左括号，即 $W(v) \leftarrow 1$ 。

情况 2.2 $W(p) = 2$ ， $W(q) = 2$ 。那么需要将 p 这个右括号与之对应的左括号 v 修改成右括号，即 $W(v) \leftarrow 2$ 。



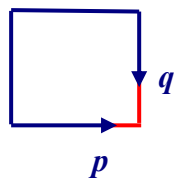
情况 2.1 图



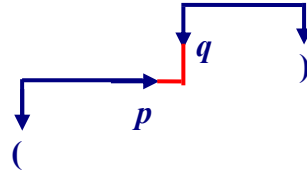
情况 2.2 图

情况 2.3 $W(p) = 1$ ， $W(q) = 2$ ，那么 p 和 q 是相对应的左括号和右括号，连接 p, q 相当于将一条路径的两端连接起来形成一个回路，这种情况下只能出现在最后一个非障碍格子。

情况 2.4 $W(p) = 2$ ， $W(q) = 1$ ，那么 p 和 q 连接起来后， p 对应的左括号和 q 对应的右括号恰好匹配，不需要修改其他的插头的状态。



情况 2.3 图



情况 2.4 图

情况 2.1, 2.2 需要计算某个左括号或右括号与之匹配的括号，这个时候需要对三进制状态解码，利用类似模拟栈的方法。因此情况 2.1, 2.2 计算新的状态的时间复杂度为 $O(n)$ ，2.3, 2.4 时间复杂度为 $O(1)$ 。

情况 3 保持原来的连通分量， $W(p)$ ， $W(q)$ 中恰好一个为 0， $W(p')$ ， $W(q')$ 中也恰好一个为 0。那么无论 p' ， q' 中哪个插头存在，都相当于是 p, q 中那个存在的插头的延续，括号性质一样，因此 $W(p') \leftarrow W(p) + W(q)$ ， $W(q') \leftarrow 0$ 或者 $W(q') \leftarrow W(p) + W(q)$ ， $W(p') \leftarrow 0$ 。计算新的状态的时间复杂度为 $O(1)$ 。

通过上面的分析可以看出，括号表示法利用了简单回路问题的“一个连通分量内只有 2 个插头”的特殊性质巧妙地用 3 进制状态存储下完整的连通信息，插

头的连通性标号相对独立，不再需要通过 $O(n)$ 扫描大范围修改连通性标号。实现的时候，我们可以用 4 进制代替 3 进制而提高程序运算效率，下面对最小表示法与括号表示法的程序效率进行比较：

表 3. 不同的状态表示的程序效率的比较

测试数据	最小表示法 7Based	最小表示法 8Based	括号表示法 3Based	括号表示法 4Based
$m = 10, n = 10$ 无障碍棋盘	31ms	15ms	0ms	0ms
$m = 11, n = 11$ (1,1)为障碍	187ms	109ms	46ms	31ms
$m = 12, n = 12$ 无障碍	873ms	499ms	265ms	140ms

可以看出，括号表示法的优势非常明显，加上它的思路清晰自然，实现也更加简单，因此对于解决这样一类简单回路问题是非常有价值的。

类似的问题还有：NWERC 2004 Pipes，Hnoi2004 Postman，Hnoi2007 Park，还有一类非回路问题也可以通过棋盘改造后用简单回路问题的方法解决，比如 POJ 1739 Tony’s Tour：给一个 $m * n$ 棋盘，有的格子是障碍，要求从左下角走到右下角，每个格子恰好经过一次，问方案总数。（ $m, n \leq 8$ ）

只需要将棋盘改造一下，问题就等价于 Formula 1 了。

.....

#.. 改造成 .#####.

... .##.#.#.

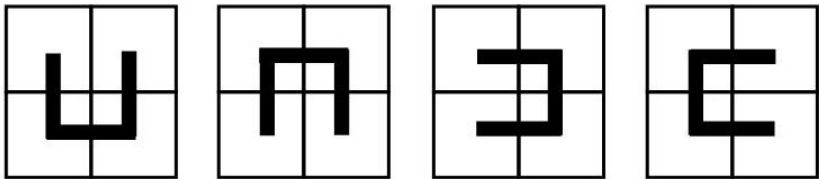
.....

介绍完简单回路问题的解法，那么一般的简单路径问题又如何解决呢？

【例 2】Formula 2⁸

问题描述

给你一个 $m * n$ 的棋盘，有的格子是障碍，要求从一个非障碍格子出发经过每个非障碍格子恰好一次，问方案总数。 $m, n \leq 10$.



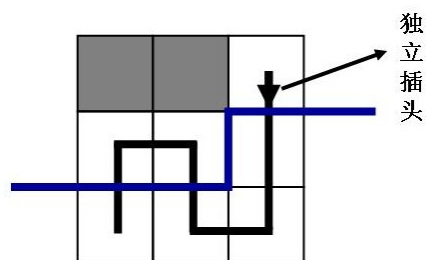
⁸ 改编自 Formula 1

如图，一个 2×2 的无障碍棋盘，共有 4 条满足要求的路径。

算法分析

确立状态：按照从上到下，从左到右依次考虑每一个格子，设 $f(i, j, S)$ 表示转移完 (i, j) 这个格子，轮廓线状态为 S 的方案总数。如果用一般的最小表示法，不仅需要记录每个插头的连通情况，还需要额外记录每个插头是否连接了路径的一端，状态表示相当复杂。依然从括号表示法这个角度来思考如何来存储轮廓线的状态：

这个问题跟简单回路问题最大的区别为：不是所有的插头都两两匹配，有的插头连接的路径的另一端不是一个插头而是整条路径的一端，我们称这样的插头为**独立插头**。不妨将原来的 3 进制状态修改成 4 进制——0 表示无插头，1 表示左括号插头，2 表示右括号插头，3 表示独立插头，这样我们就可以用 4 进制完整地记录下轮廓线的信息，图中状态表示为 $(1203)_4$ 。



状态转移：依然设 $(i, j-1)$ 的右插头为 p ， $(i-1, j)$ 的下插头为 q ， (i, j) 的下插头为 p' ，右插头为 q' 。部分转移同简单回路问题完全一样，这里不再赘述，下面分三类情况讨论与独立插头有关的转移：

情况 1 $W(p) = 0, W(q) = 0$ 。当前格子可能成为路径的一端，即右插头或下插头是独立插头，因此 $W(p') \leftarrow 3, W(q') \leftarrow 0$ 或者 $W(q') \leftarrow 3, W(p') \leftarrow 0$ 。

情况 2 $W(p) > 0, W(q) > 0$ ，那么 $W(p') \leftarrow 0, W(q') \leftarrow 0$

情况 2.1 $W(p) = 3, W(q) = 3$ ，将插头 p 和 q 连接起来就相当于形成了一条完整的路径，这种情况只能出现在最后一个非障碍格子。

情况 2.2 $W(p), W(q)$ 中有一个为 3，如果 p 为独立插头，那么无论 q 是左括号插头还是右括号插头，与 q 相匹配的插头 v 成为了独立插头，因此， $W(v) \leftarrow 3$ 。如果 q 为独立插头，类似处理。

情况 3 $W(p), W(q)$ 中有一个 > 0 ，即 p, q 中有一个插头存在。

情况 3.1 如果这个插头为独立插头，若在最后一个非障碍格子，这个插头可以成为路径的一端，否则可以用右插头或下插头来延续这个独立插头。

情况 3.2 如果这个插头是左括号或右括号，那么我们以将这个插头“封住”，使它成为路径的一端，需要将这个插头所匹配的另一个插头的状态修改成为独立插头。

情况 2.2, 3.2 需要计算某个左括号或右括号与之匹配的括号，计算新的状态

的时间复杂度为 $O(n)$ ，其余情况计算新的状态的时间复杂度为 $O(1)$ 。

特别需要注意，任何时候轮廓线上独立插头的个数不可以超过 2 个。至此问题完整解决， $m = n = 10$ 的无障碍棋盘，扩展的状态总数为 3493315，完全可以承受。

上面两类题目我们用括号表示法取得了很不错的效果，但是它存在一定的局限性，即插头必须满足两两匹配。那么对于更加一般的问题，一个连通分量内出现大于 2 个插头，上述的括号表示方法显得束手无策。下面将介绍一种括号表示法的变形，它可以适用于出现连通块内大于 2 个插头的问题，我们称之为**广义的括号表示法**：

假设一个连通分量从左到右有多个插头，不妨将最左边的插头标记为 “(”，最右边的插头标记为 “)”，中间的插头全部标记为 “)(”，那么能够匹配的括号对应的插头连通。如果问题中可能出现一个连通分量只有一个插头，那么这个插头标记为 “()”，这样插头之间的连通性可用括号序列完整地记录下来，比如对于一个连通性状态为 $\{1, 2, 2, 3, 4, 3, 2, 1\}$ ，我们可以用 $((-)(-(-()--))$ 记录。

这种广义的括号表示方法需要用 4 进制甚至 5 进制存储状态，而且直接对状态连通性进行修改情况非常多，最好还是将状态进行解码，修改后再重新编码。下文我们将会运用广义的括号表示法解决一些具体的问题。

小结

本章针对一类简单路径问题，提出了一种新的状态表示方法——括号表示法，最后提出了广义的括号表示方法。相比普通的最小表示法，括号表示法巧妙地把连通块与括号匹配一一对应，使得状态更加简单明了，虽然不会减少扩展的状态总数，但是转移开销的常数要小很多，是一个不错的方法。

三. 一类棋盘染色问题

有一类这样的问题——给你一个 $m * n$ 的棋盘，要求给每个格子染上一种颜色(共 k 种颜色)，每种颜色的格子相互连通 (4 连通)。本章主要对这类问题的解法进行探讨，我们从一个例题说起：

【例 3】Black & White⁹

⁹ Source : Uva10572

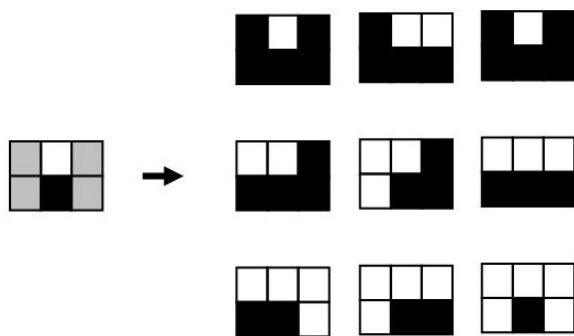
问题描述

一个 $m * n$ 的棋盘，有的格子已经染上黑色或白色，现在要求将所有的未染色格子染上黑色或白色，使得满足以下 2 个限制：

- 1) 所有的黑色的格子是连通的，所有的白色格子也是连通的。
- 2) 不会有一个 $2 * 2$ 的子矩阵的 4 个格子的颜色全部相同。

问方案总数. ($m, n \leq 8$)

如下图， $m = 2, n = 3$ ，灰色格子为未染色格子，共有 9 种染色方案。



算法分析

这是一个典型的棋盘染色问题，着色规则有：

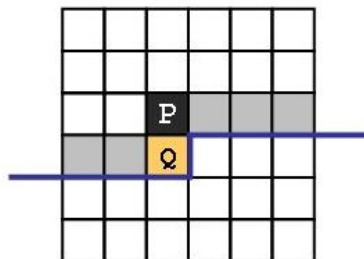
- 1) 只有黑白两种颜色，即 $k = 2$ ，并且同色的格子互相连通。
- 2) 没有同色的 $2 * 2$ 的格子。

对于简单路径问题来说，相邻的格子是否连通取决于它们之间的插头是否存在，状态记录轮廓线上每个插头是否存在以及插头之间的连通性；而棋盘染色问题相邻的格子是否连通取决于它们的颜色是否相同，这就需要记录轮廓线上方 n 个格子的颜色以及格子之间的连通性。

确立状态 设当前转移完 $Q(i, j)$ 这个格子，对以后的决策产生影响的信息有：轮廓线上方 n 个格子的染色情况以及它们的连通性，由第 2 条着色规则“没有同色的 $2 * 2$ 的格子”可知 $P(i-1, j)$ 的颜色会影响到 $(i, j+1)$ 着色，因此我们还需要额外记录格子 P 的颜色。动态规划的状态为：

$f(i, j, S_0, S_1, cp)$ 表示转移完 (i, j) ，轮廓线上从左到右 n 个格子的染色情况为 S_0 ($0 \leq S_0 < 2^n$)，连通性状态为 S_1 ，格子 P 的颜色为 cp (0 或 1) 的方案总数。

状态的精简 如果相邻的 2 个格子不属于同一个连通块，那么它们必然不同色，因此只需要记录 $(i, 1)$ 和 $(i-1, j+1)$ 两个格子的颜色，利用 S_1 就可以推出 n 个格子的颜色。这个



精简不会减少状态的总数，仍然需要一个变量来记录两个格子的颜色，因此意义并不大，这里只是提一下。

状态转移 枚举当前格子 (i, j) 的颜色，计算新的状态： S_0 和 cp 都很容易 $O(1)$ 计算出来。考虑计算 S_1 ：轮廓线的变化相当于将记录 $(i-1, j)$ 的连通性改成记录 (i, j) 的连通性。根据当前格子与上面的格子和左边的格子是否同色分四类情况讨论。应当注意的是如果 (i, j) 和 $(i-1, j)$ 不同色，并且 $(i-1, j)$ 在轮廓线上为一个单独的一个连通块，那么 $(i-1, j)$ 以后都不可能与其他格子连通，即剩余的格子都必须染上与 $(i-1, j)$ 相反的颜色，需要特殊判断。转移的时间复杂度为 $O(n)$ 。计算新状态的 S_1 程序框架如下：

```

将前一个状态的  $S_1$  解码，连通性存入  $c[1], c[2], \dots, c[n]$ .
If  $(i, j)$  与  $(i-1, j)$  不同色并且  $(i-1, j)$  为一个单独的连通块 Then
    特殊判断
Else
    If  $(i, j)$  与  $(i-1, j)$  和  $(i, j-1)$  均同色 Then
        For  $k \leftarrow 1$  to  $n$ 
            If  $c[k] = c[j]$  Then
                 $c[k] \leftarrow c[j-1]$  // 合并两个连通块
            EndIf
        EndIf
    Else
        If  $(i, j)$  与  $(i-1, j)$  和  $(i, j-1)$  均不同色 Then
             $c[j] \leftarrow$  最大可能出现的连通块标号 //  $(i, j)$  新建一个连通块.
        Else
            If  $(i, j)$  与  $(i, j-1)$  同色与  $(i-1, j)$  不同色 Then
                 $c[j] \leftarrow c[j-1]$  //  $(i, j)$  的连通性标号跟  $(i, j-1)$  相同.
            EndIf
        EndIf
    EndIf
EndIf
EndIf
对  $c[]$   $O(n)$  扫描，修改成最小表示，利用  $c[]$  编码计算出新的  $S_1$ .

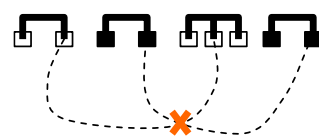
```

对于 $m = n = 8$ 的一个全部未染色的棋盘，扩展出来的状态总数为122395，转移需要时间为 $O(n)$ ，因此总的时间复杂度为 $O(\text{TotalState} * n) = 979160$ ，运行时间 $< 0.1s$ 。至此问题完整解决。类似可以解决的问题还有2007年重庆市选拔赛Rect和IPSC 2007 Delicious Cake。

扩展 上面提到的是4连通问题，如果要求8连通呢？

4连通问题是两个格子至少有一条边重合为连通，而8连通问题是两个格子至少有一个顶点重合为连通，因此需要记录所有至少有一个顶点在轮廓线上的格子的连通和染色情况，即包括 $(i-1, j)$ 在内的 $n+1$ 个格子。

一个优化的方向 扩展的状态中无效状态的总数很大程度上决定了算法的



效率。比如 **Black & White** 中如果出现右图的状态，那么无论之后如何决策，都不可能满足同色的格子互相连通的性质，因此它是一个无效状态。对于任何一个 k 染色棋盘问题，如果从左到右有 4 个相互不嵌套¹⁰的连通块 a, b, c, d ， a, c 同色， b, d 同色且与 a, c 不同色，那么这个状态为无效状态。

小结

本章介绍了解决一类棋盘染色问题的一般思路。无论染色规则多么复杂，我们只要在基本状态即“轮廓线上方与其相连的格子的连通性以及染色情况”的基础上，根据题目的需要在状态中增加对以后的决策可能产生影响的信息，问题都可以迎刃而解了。

四. 一类基于非棋盘模型的问题

本章将会介绍一类基于非棋盘模型的连通性状态压缩动态规划问题，它虽然不具有棋盘模型的特殊结构，但是解法的核心思想又跟棋盘模型的问题有着异曲同工之处。

【例 4】生成树计数¹¹

问题描述

给你一个 n 个点的无向连通图，其边集为：任何两个不同的点 $i, j (1 \leq i, j \leq n)$ ，如果 $|i - j| \leq k$ ，那么有一条无向边 $\langle i, j \rangle$ 。已知 n 和 k ，求这个图的生成树个数。
 $n \leq 10^{15}$, $2 \leq k \leq 5$ 。

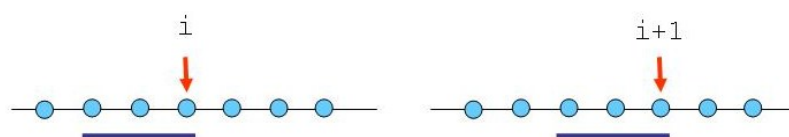
算法分析

这个题给我们的第一印象是： n 非常大， k 却非常小。

生成树最重要的两个性质：无环，连通。那么如果按照 $1, 2, \dots, n$ 的顺序依次考虑每一个点与前面的哪些点相连，并且保证任何时候都不会出现环，最后统计所有的点全部在一个连通分量内的方案总数即为最终的答案。

在棋盘模型的问题中，我们提出了轮廓线这个概念，任何时候只有轮廓线上方与其直接相连的格子对以后的决策会产生影响。类似地我们分析一下这个问题，当我们确定了 $1 \sim i$ 的所有点的连边情况后，哪些信息对以后的决策会产生影响： $1 \sim i - k$ 这些点与 i 之后的点一定没有边相连，那么对 i 以后的点的决策不会产生直接的影响，因此我们需要记录的仅仅是 $i - k + 1 \sim i$ 这 k 个点的连通信息！

如下图，我们不妨也称蓝线为轮廓线，因为只有轮廓线上的点的信息会对轮廓线右边的点的决策产生直接的影响。这样我们就很容易**确立状态**：



设 $f(i, S)$ 表示考虑完前 i 个点的连边情况后， $i - k + 1 \dots i$ 这 k 个点的连通情况为 S 。

¹⁰ “嵌套”的概念可以用广义的括号匹配的表示方法来理解

¹¹ Source : Noi2007 Day2 生成树计数, Count

转移状态： $O(2^k)$ 依次枚举点 i 与 $i-1, \dots, i-k$ 这 k 个点是否相连。转移的时候需要注意： $i-1, \dots, i-k$ 这 k 个点，任何一个连通块， i 最多只能与其中的一个点相连，这样可以避免环的出现。如果 $i-k$ 在轮廓线上为一个单独的连通块，那么 i 必然与 $i-k$ 相连，这样可以避免出现孤立的连通块。比如对于一个 $k=5$ 的状态 $f(i-1, \{1,2,2,1,3\})$ 来说，如果点 i 与 $i-2$ 和 $i-1$ 相连，那么新的状态为 $f(i, \{1,1,2,2,2\})$ 。这样我们就可以在 $O(2^k \cdot k)$ 的时间复杂度内完成状态的转移。

算法实现：设 T_k 表示 k 个点的本质不同的连通情况的个数，搜索可知 $T_5=52$ 。动态规划的时间复杂度为 $O(n \cdot T_k \cdot 2^k \cdot k)$ ，依然太大。可以发现当 $i \geq k$ ，状态 $f(i, S)$ 是否可以转移到 $f(i+1, S')$ 只与 S, S' 有关，这样我们就可以用矩阵乘法实现动态规划加速，由于这不是本文的重点，这里不再详细介绍。最终的时间复杂度为 $O(T_k^3 \cdot \log_2 n)$ ，对于 $k=5, T_k=52$ 的数据规模来说已经完全可以承受了，至此问题完整解决。

本题中的无向图非常特殊，每个点只和距离它不超过 k 的点有边相连，并且 k 非常小。对于棋盘模型的问题，可以抽象成一个特殊的无向图—— $m \cdot n$ 个点，每个点只与它上下左右四个点有边相连。那么对于一个与连通性有关的无向图问题，无向图具备怎样的特点才可以用基于状态压缩的动态规划来解决？分析以上几个问题，不难发现它们有一个共同点：给无向图中的点找一个序，在这个序中有边相连的两个点的距离不超过 p (p 很小)，这样我们就可以以当前决策完序中前 i 个，最后 p 个点的连通性为状态作动态规划。棋盘模型的问题中序即为从上到下，从左到右或从右到左，从上到下， p 为 m 或 n ，因此棋盘模型的问题 m 和 n 中至少有一个数会非常小。

小结

本章写得比较简略，但是依然能够给我们很多的启示。处理这样的一类非棋盘模型的问题，一般的思路是寻找某一个序依次考虑每个点的决策，并分析哪些信息对以后的决策会产生影响，找到问题中的“轮廓线”，以轮廓线的信息来确立动态规划的状态。通常来说，轮廓线上的信息比较少，这也是能够作状态压缩动态规划的基础，像本题中 $k \leq 5$ 这样的条件往往能成为解决问题的突破口。

五. 一类最优性问题的剪枝技巧

基于连通性状态压缩的动态规划问题的算法的效率主要取决于状态的总数和转移的开销，减少状态总数和降低转移开销成为了优化的核心内容。前面的章节我们提到了一些优化的技巧，这一章我们选取了一个非常有趣的题目 **Rocket Mania** 来介绍针对这样的一类最优性问题，如何通过剪枝使状态总数大大减少而提高算法效率。

【例 5】Rocket Mania¹²

¹² Source : Zju 2125, Online Contest of Fantasy Game

问题描述

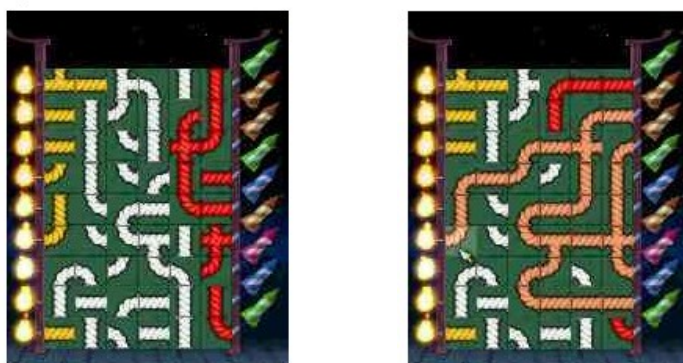
这个题目的背景是幻想游戏的“中国烟花”：

给你一个 9×6 的棋盘，棋盘的左边有 9 根火柴，右边有 9 个火箭．棋盘中的每一个格子可能是一个空格子也可能是一段管道，管道的类型有 4 种：



一个火箭能够被发射当且仅当存在一条由管道组成的从一根点燃的火柴到这个火箭的路径．

给你棋盘的初始状态以及 X ，你的目标是旋转每个格子内的管道 $0, 90, 180$ 或 270 度，使得当点燃左边第 X 根火柴后，被发射的火箭个数尽可能多．



算法分析

确立状态：按照从左到右，从上到下的顺序依次考虑每一个格子，我们需要记录每个插头是否已经点燃以及它们之间的连通情况．因此状态为 $f(i, j, S, fired)$ 表示转移完 (i, j) ，轮廓线上 10 个插头的连通性为 S (把每个插头是否存在记录在 S 中)，10 个插头是否被点燃的 2 进制数 $fired$ 的状态能否达到．

那么最后的答案为所有可以达到的状态 $f(9, 6, S, fired)$ 中 $Ones[fired]$ 的最大值，其中 $Ones[x]$ 表示二进制数 x 的 1 的个数．

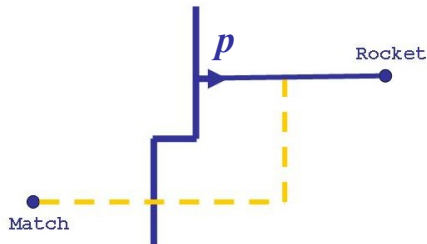
状态转移：依次枚举每一个格子的旋转方式(最多 4 种)，根据当前格子是否可以与上面的格子和左边的格子通过插头连接起来分情况讨论， $O(m)$ 扫描计算出新的状态．前面的题目我们已经很详细地介绍过棋盘模型的问题的转移方法，这里不再赘述．

如果直接按照上面的思路作动态规划，Sample 也需要运行 $> 60s$ ，实在令人无法满意．优化，势在必行．如何通过剪枝优化来减少扩展的状态总数，尽可能舍去无效状态成为了现在所面临的问题：

剪枝通常可以分为两类：一．可行性剪枝，即将无论之后如何决策，都不可能满足题目要求的状态剪掉；二．最优性剪枝，即对于最优性问题，将不可能成为最优解的状态给剪掉．我们从这两个角度入手来考虑问题：

剪枝一：如果轮廓线上所有的插头全部都未被点燃，那么最后所有的火箭都不可能发射，所以这个状态可以舍去．这个剪枝看上去非常显然，对于大部分数据却可以剪掉近乎一半的状态．

剪枝二：如果轮廓线上有一个插头 p ，它没有被火柴点燃且没有其它的插头与它连通，那么这个插头可以认为是“无效”插头。因为即使这个插头所在的路径以后会被点燃而可以发射某个火箭，那么一定存在另一条路径可以不经过这个插头而发射火箭，如图。这种情况下将插头设置为不存在。这是最重要的一个剪枝，大部分数据的状态总数可以缩小七八倍，甚至十几倍。



剪枝三：这是一个最优性问题，我们考虑**最优性剪枝：**对于一个格子 (i, j) 的两个状态

$(S_0, Fired_0)$, $(S_1, Fired_1)$ ，如果第一个状态的

每一个存在的插头在第二个状态中不仅存在而且都被点燃，那么无论以后如何决策，第二个状态点燃的火箭个数不会少于第一个状态，这样我们就可以果断地舍去第一个状态。对于每一个 (i, j) ，选择 $Ones[Fired]$ 最多的一个状态 $Best$ ，如果一个状态一定不比 $Best$ 好，就可以舍去。

剪枝四：从边界情况入手，边界状态非常特殊，也非常容易导致产生无效状态。分析一下，转移完最后一列的某个格子 $(i, 6)$ 后，如果 I 类插头中某个插头 p 没有被点燃，并且 II 类插头中没有插头与它连通，那么这个插头就成了“无效”插头。

比较以上四种剪枝的效果，由于不同的棋盘初始状态扩展的状态总数差异较大，因此选取 10 组不同的棋盘初始状态来测试扩展状态的总数。10 组数据大致分布如下：Test 1~4 依次为全部“—”，“L”，“T”，“+”，Test 5 为奇数行“L”，偶数行为空，Test 6 为“L”，“T”交替。Test 7~10 为随机数据，“L”，“T”分布较多，Test 10 的“—”较多¹³。

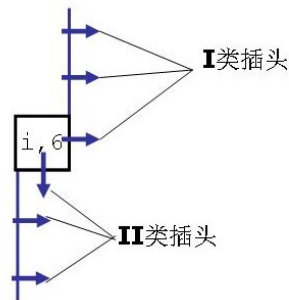


表 4. 四种剪枝扩展的状态总数的比较

Test	无剪枝	剪枝 1	剪枝 1,2	剪枝 1,2,3	剪枝 1,2,3,4
1	105	55	55	55	55
2	304250	121585	8758	8758	7620
3	18193954	13870994	1133036	11451	11451
4	55	55	55	55	55
5	958	2	2	2	2
6	4409663	2914156	438714	155049	153665
7	1662550	865193	137771	67473	67086
8	1697106	1075240	156778	7741	7741
9	557590	379853	75449	24856	13370
10	210290	43759	6084	6079	2957

由上表可以看出，优化后扩展的状态总数已经非常少了，剪枝的效果非常明显。我们从可行性和最优性两个角度，从一般情况和边界情况入手提出了 4 种剪枝方法，虽然有的剪枝看上去微不足道，但是它产生的效果确是惊人的。当然剪

¹³ 具体测试数据请参加附录

枝方法远远不止这 4 种，只要抓住问题的特征不断分析，就可以提出更多更好的剪枝方法。

值得一提的是 S 的状态表示，如果用普通的最小表示法，需要用 10 进制存储状态。由剪枝 2 可知如果一个插头属于一个单独的连通块，那么它一定被火柴点燃。如果使用广义的括号表示法，可以将无插头状态和单独的连通块插头都有 “()” 表示，利用 *fired* 来区别，这样就可以用 “(”, “)”, “)(”, “()” —— 4 进制完整记录下 n 个格子的连通性，相比 10 进制有一定的常数优势。

至此，问题完整解决，60 多组测试数据运行时间 < 1.5s，实际效果确实不错。

这个问题还有一个加强版¹⁴：跟本题唯一不同的是，左边所有的火柴全部点燃，那么只要把初始状态中 9 个右插头全部设置为点燃，且为一个连通块即可。

小结

本章我们以 RocketMania 一题为例介绍了解决一类最优性的连通性状态压缩动态规划问题的剪枝技巧，从可行性和最优性这两个角度出发而达到减少状态总数的目的。在解题的过程中，要抓住问题的主要特征，多思考，多尝试，才能做的越来越好，优化是无止境的。

六. 总结

本文立足于基于连通性状态压缩动态规划问题的解法和优化两个方面。

全文介绍了基于连通性状态压缩的动态规划问题的一般解法及其相关概念；针对一类特殊的问题——简单回路和简单路径问题，提出了括号表示法以及括号表示法的改进，最后从特殊问题回归到一般问题，提出了广义的括号表示法，这是文章的核心内容；接着对于一类棋盘染色问题和基于非棋盘模型的问题的解法作一个探讨；最后我们把重点放在了剪枝优化上，结合一个非常有趣的例题谈针对这类动态规划问题剪枝的重要性。

当然本文不可能涵盖基于连通性状态压缩动态规划问题的方方面面，因此关键是要掌握解决问题的思路，在解题的过程中抓住问题的特征，深入分析，灵活运用。从上面的例题中可以发现，细节是不可忽略的因素，它很大程度上决定了算法的效率。因此平时我们要养成良好的编程习惯，注意细节，注重常数优化。做到多思考，多分析，多实验，不断优化，精益求精。让我们做得越来越好！

【参考文献】

- 【1】 《算法艺术与信息学竞赛》 刘汝佳、黄亮
- 【2】 金恺 2004 年国家集训队作业 《Black & White》解题报告
- 【3】 毛子青 2001 年国家集训队论文 《动态规划算法的优化技巧》
- 【4】 Uva 在线题库: <http://icpcres.ecs.baylor.edu/onlinejudge>

¹⁴Zju 2126 Rocket Mania Plus

【5】 Ural 在线题库: <http://acm.timus.ru>

【6】 Zju 在线题库: <http://acm.zju.edu.cn>

【感谢】

感谢朱全民老师在我写这篇论文时对我的指导.

感谢刘汝佳教练对我的指导和启发.

感谢广西柳铁一中的刘宸亨同学和江苏省常州高级中学的金斌同学对我的论文提供了很大的帮助.

感谢集训队员郑瞰, 周冬, 余林韵, 俞华程, 顾研, 周梦宇, 肖汉骏对我的帮助.

【附录】

附录一. 第五章中的实验测试数据



RocketMania.in

附录二. 英文原题



Formula 1.mht



Black and White.mht



RocketMania.mht